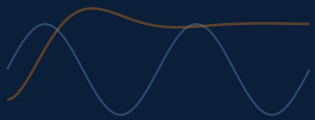
WEEK 2 · DAY 5 / 5

Mechatronics and Robotics Engineering (50 h)



Sensors, Measurement and System Integration

6.4 Machine vision ▪ 6.5 Sampling ▪ 6.6 Signal processing ▪ 6.7 Communication ▪ 6.8 Case study & course wrap-up

Mechatronics and Robotics Engineering Programme

Week 2 · Day 5 ▪ 2 weeks · 10 days · 50 hours

Agenda and learning outcomes

Code	Session	Time
6.4	Machine vision in robotics	60 min
6.5	Sampling and digitisation	60 min
6.6	Signal processing and filters	60 min
6.7	Communication and integration	60 min
6.8	Integration case study & course wrap-up	60 min

5 h net teaching time; 10-min breaks roughly every hour are extra.

By the end of today you can . . .

- project a 3D point into a camera image
- explain calibration and hand–eye setup
- apply the Nyquist rule and size an ADC
- write a one-line digital low-pass filter
- explain how a complementary filter fuses sensors
- choose a network for each link in a robot cell
- build a latency and error budget for a cell

What we learned yesterday (W2D4)

5.6 Conveyors, AGV/AMR, FMS cells, cobots

5.7 Safety: risk assessment, ISO 10218, PL, light curtains

6.1 Accuracy, precision, resolution, range, linearity

6.2 Encoders, resolvers, IMU

6.3 Force/torque, proximity, range, LiDAR

Today From the sensor signal to a working robot cell

Three ideas we reuse today

- **Resolution** is the smallest step a sensor can show; **accuracy** is how close it is to the truth.
- An **IMU** gives angular rate (gyro) and acceleration — both imperfect.
- A **homogeneous transform** T moves points between frames (W2D1).

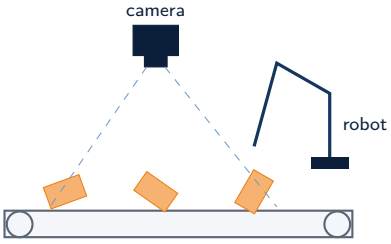
Today: the robot's *eyes* (vision), how a signal becomes *numbers*, how we *clean* them, and how all devices *talk*.

Session 1

6.4 Machine vision in robotics

Why robots need eyes

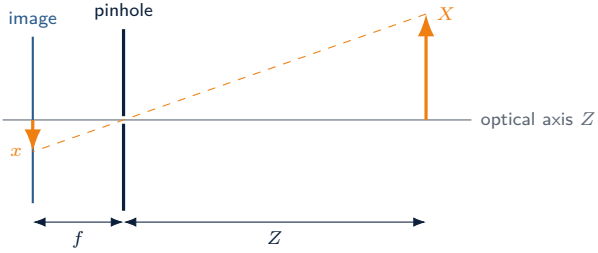
- A taught robot repeats a path blindly: parts must always be in the **same place**.
- A camera tells the robot *where* the part is, *which* part it is, and whether it is *good*.
- Typical tasks: **locate** (pick from a conveyor or bin), **inspect** (defects, presence), **read** (barcodes), **guide** (seam tracking in welding).
- **2D vision**: one camera, parts on a known plane. **3D vision**: stereo, structured light or time-of-flight.





Industrial machine-vision camera: Ginkgomaier, CC BY-SA 4.0, via Wikimedia Commons

Refresher: the pinhole camera



- Like a dark room with a small hole: the scene appears upside down on the back wall.
- Similar triangles give the size in the image.

$$u = f_x \frac{X}{Z} + c_x, \qquad v = f_y \frac{Y}{Z} + c_y$$

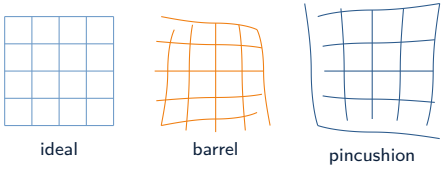
X, Y, Z [m]: point in the camera frame; f_x, f_y [px]: focal length in pixels; c_x, c_y [px]: image centre; u, v [px]: pixel.

Worked example

1. $f_x = 1000$ px, point at $X = 0.10$ m, $Z = 0.50$ m
2. $u - c_x = 1000 \cdot 0.10/0.50 = 200$ px
3. One pixel at $Z = 0.5$ m covers $0.5/1000 = 0.5$ mm
4. Same object at 1 m: half the pixels

The camera model: intrinsics and lens distortion

$$\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} \sim \underbrace{\begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}}_{K \text{ (intrinsics)}} \begin{bmatrix} X/Z \\ Y/Z \\ 1 \end{bmatrix}$$



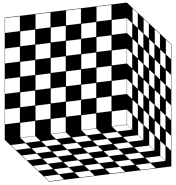
- **Intrinsic** parameters: belong to the camera + lens (f_x, f_y, c_x, c_y, distortion).
- **Extrinsic** parameters: where the camera is, a transform T like in W2D1.
- Real lenses bend straight lines: **radial distortion**, $r_d = r\,(1 + k_1 r^2 + k_2 r^4)$.

The symbol $\sim$ means "equal up to a scale factor": all points on one ray through the pinhole land on the same pixel. **One camera cannot see depth.**

Camera calibration: measuring K and the distortion

1. Print a **checkerboard** with known square size (e.g. 25 mm).
2. Take 15–30 images from different angles and distances.
3. Software finds the corners (sub-pixel accurate).
4. It searches K, k_1, k_2 and each board pose so that projected corners match the found ones.
5. Quality number: **reprojection error**, typically < 0.3 px.

Going further: Zhang's method (2000) solves this as a linear estimate followed by non-linear least squares; available in OpenCV and MATLAB.

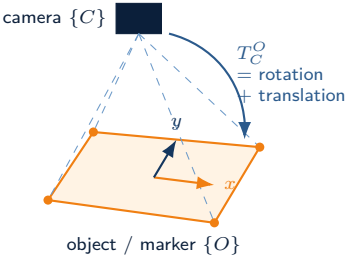


Three-plane chessboard calibration target: Brian moore81, CC BY-SA 4.0, via Wikimedia Commons

Analogy

Like tuning a guitar with a reference tone: we show the camera a pattern whose geometry we know exactly and correct the camera until it "sounds right".

Pose estimation: where is the object?

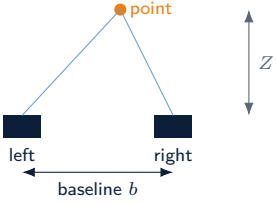


- If we know ≥ 4 points on the object (a CAD model or a printed marker) and see them in the image, we can solve for its **pose** T_C^O.
- This is the **Perspective-n-Point** (PnP) problem.
- **Fiducial markers** (ArUco, AprilTag) are black-and-white squares designed to be found and identified quickly.



marker: border + ID code

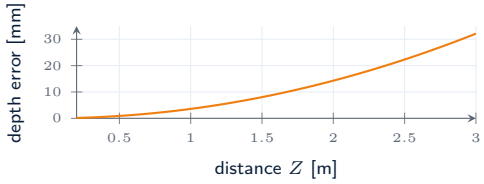
Seeing depth: stereo, structured light, time of flight



$$Z = \frac{f\,b}{d} \qquad d = u_L - u_R \text{ (disparity, px)}$$

Worked example

$f = 700$ px, $b = 0.10$ m, $d = 35$ px $\Rightarrow Z = 700 \cdot 0.10/35 = 2.0$ m



Error $\Delta Z = Z^2 \Delta d/(fb)$ for $\Delta d = 0.25$ px: grows with Z^2.

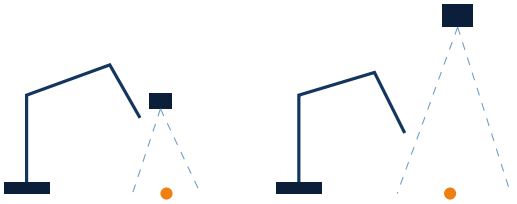
Active



- **Structured light**: project a known pattern, measure how it bends.
- **Time of flight**: measure the light's travel time (like LiDAR).

stereo depth camera: Marc Auledas, CC BY-SA 4.0, via Wikimedia Commons

Hand–eye calibration: where is the camera on the robot?



eye-in-hand
camera on the flange

eye-to-hand
camera fixed in the cell

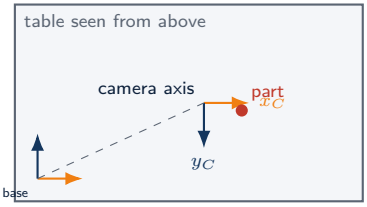
$$T_B^O = T_B^E \; T_E^C \; T_C^O$$

base→flange (robot knows it) × flange→camera (**unknown**) × camera→object (vision).

- The camera reports the object in *its own* frame; the robot needs it in the *base* frame.
- The missing link T_E^C is found by **hand–eye calibration**: move the robot to 10–20 poses and look at a fixed checkerboard each time.
- Eye-to-hand: the unknown is T_B^C instead.

Going further: this leads to the classic matrix equation $AX = XB$ (Tsai–Lenz 1989, Park–Martin 1994).

Worked example: from pixel to robot target



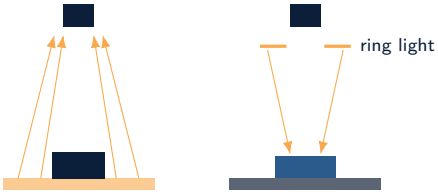
Fixed camera looks straight down from 0.5 m; its axis hits the table at $(600, 0)$ mm in base coordinates; image y points along base $-y$.

Step by step

1. Image centre $(c_x, c_y) = (640, 480)$; part found at $(u, v) = (840, 520)$ px.
2. Offset $(200,\ 40)$ px $\times$ 0.5 mm/px $=$ $(100,\ 20)$ mm in the camera frame.
3. Rotate into base: $x_B = +100$, $y_B = -20$ mm.
4. Add the camera position: target $=$ $(700,\ -20)$ mm.

- Steps 3–4 are just T_B^C applied to a point (W2D1).
- A 1 px error is 0.5 mm at the gripper.

Practical vision: light, contrast and speed



Machine-vision system: European Editors, CC BY-SA 4.0, via Wikimedia Commons

- Good **lighting** solves more vision problems than clever software.
- Moving parts blur: exposure 1 ms at 0.5 m/s = 0.5 mm of smear.
- Image processing takes time (20 ms to 200 ms): the result is **old** when it arrives.
- Trigger the camera with a signal and **time-stamp** the image (more in 6.8).

Going further: deep-learning detectors handle cluttered bins and varying parts but need training data and a GPU.

Check your understanding: machine vision

Check your understanding

1. A camera with $f_x = 800$ px looks at a part 0.4 m away. How many millimetres does one pixel cover?
2. Why can a single calibrated camera not tell the height of a part on its own?
3. A stereo camera has an error of 5 mm at 1 m. Roughly what error do you expect at 2 m?

Summary and key terms: machine vision

- Pinhole model: $u = f_x X/Z + c_x$; one camera loses depth.
- Calibration with a checkerboard finds K and lens distortion.
- Pose estimation (PnP, markers) gives T_C^O.
- Hand–eye calibration links camera and robot: $T_B^O = T_B^E T_E^C T_C^O$.
- Stereo: $Z = fb/d$, error grows with Z^2. Light the scene well.

Key terms

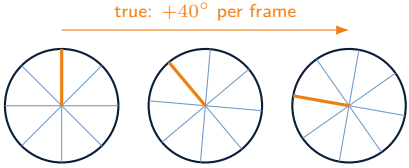
intrinsics K: focal length, image centre
extrinsics camera pose T
distortion lens bends straight lines
reprojection error calibration quality [px]
PnP pose from known points
disparity left–right pixel shift

[Corke, 2017]; [Szeliski, 2022]; [Zhang, 2000].

Session 2

6.5 Sampling and digitisation

Refresher: sampling and the wagon-wheel effect



spokes repeat every 45°: the eye sees -5° per frame $\Rightarrow$ wheel seems to turn **backwards**

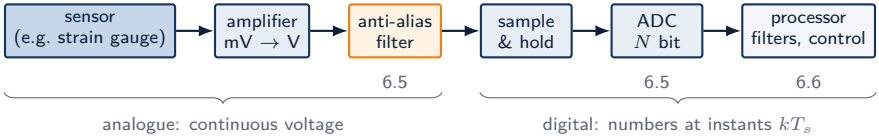
Refresher: sampling

A computer only sees the signal at instants $t = kT_s$; the **sampling frequency** is $f_s = 1/T_s$. Like a film camera taking 24 pictures per second.

Nyquist: $f_{\text{signal}} < \dfrac{f_s}{2}$

- Faster signals are not lost, they are **disguised** as slower ones: **aliasing**.
- Same as a wagon wheel in a film or a strobe light.

The measurement chain, and a refresher on binary numbers

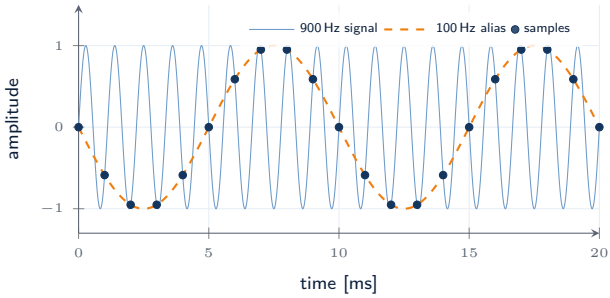


Refresher: binary numbers

N bits can count 2^N different values. $1011_2 = 8 + 0 + 2 + 1 = 11$. 3 bits → 8 levels, 12 bits → 4096, 16 bits → 65 536.

- Two things are lost on the way: values **between samples** (time) and values **between steps** (amplitude).
- Both errors are small if we choose f_s and N well.

Aliasing in a measured signal

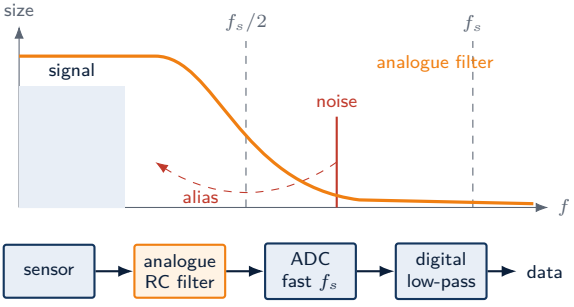


Example

1. $f_s = 1\,\text{kHz}$, so $f_s/2 = 500\,\text{Hz}$
2. Signal at $900\,\text{Hz} > 500\,\text{Hz}$: too fast
3. It appears at $|900 - 1000| = 100\,\text{Hz}$

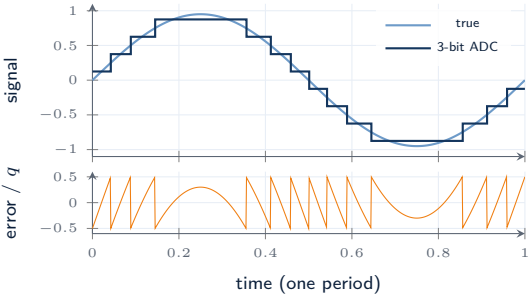
- Rule: $f_{\text{alias}} = |f - k\,f_s|$, nearest whole k.
- After sampling, the fake 100 Hz looks exactly like a real one: **no digital filter can remove it**.

Anti-aliasing: analogue before the ADC, digital after



- Only an **analogue** filter in front of the ADC can stop aliasing (see wagon wheel).
- Sampling much faster than needed (**oversampling**) lets a simple RC do the job; the sharp filtering is then done digitally.
- The analogue filter also adds delay: keep its corner ≥ 5–$10\times$ the loop bandwidth.

Quantisation: turning a voltage into a number



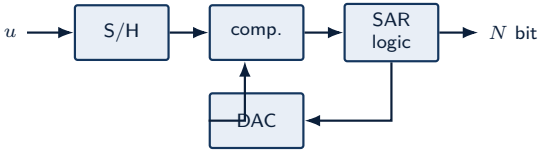
- An N-bit **ADC** (analogue-to-digital converter) has 2^N steps; step size $q = \text{range}/2^N$.
- Rounding gives an error of at most $\pm q/2$: **quantisation noise**.

$$\text{SNR} \approx 6.02\,N + 1.76 \text{ dB}$$

Example

1. Joint torque sensor, range -100 to 100 Nm $= 200$ Nm, 12 bit
2. $q = 200/4096 = 0.049$ Nm
3. SNR $\approx 6 \cdot 12 + 1.8 = 74$ dB; each extra bit $+6$ dB

ADC types: SAR and sigma-delta



SAR

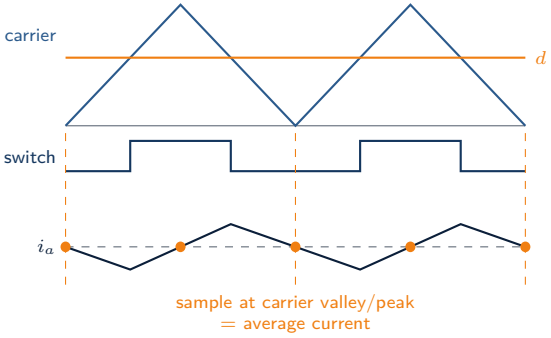
Is it above half range? Yes → bit 1. Now above $3/4$? ... One bit per guess: 12 guesses for 12 bits.

	SAR	Sigma-delta
Resolution	12–16 bit	16–24 bit
Delay	very short	longer
Typical use	phase currents	load cells, isolated current

- **S/H** (sample-and-hold) freezes the voltage during conversion.
- **Sigma-delta**: a very fast 1-bit converter, averaged to many bits.

Going further: $\Sigma\Delta$ modulators use oversampling and noise shaping; a sinc^3 decimation filter turns the bit stream into words.

Sampling in step with the PWM, and delay



- The PWM makes the motor current ripple (W1D4).
- Sampling exactly in the middle of a switching pattern gives the **average** current: no ripple, no aliasing.
- Every delay costs phase:

$$\Delta\varphi = 360^\circ \cdot f \cdot T_d$$

Example

1. $f_s = 16\,\text{kHz}$, delay $T_d = 1.5\,T_s = 94\,\mu\text{s}$
2. At $f = 1\,\text{kHz}$: $360{\cdot}1000{\cdot}94{\cdot}10^{-6} = 34^\circ$

Check your understanding: sampling

Check your understanding

1. A 9 kHz vibration is sampled at $f_s = 10$ kHz. At which frequency does it appear in the data?
2. How much better is the ideal SNR of a 16-bit ADC than a 12-bit one?
3. Why must the anti-aliasing filter be analogue, i.e. placed *before* the ADC?

Summary and key terms: sampling and digitisation

- Sample faster than twice the highest frequency (Nyquist).
- Too-fast content folds back as a fake slow signal (aliasing).
- N bits: step $q = \text{range}/2^N$, SNR $\approx 6N$ dB.
- SAR for speed, sigma-delta for resolution.
- Sample in sync with the PWM; every delay costs $360^\circ f T_d$.

Key terms

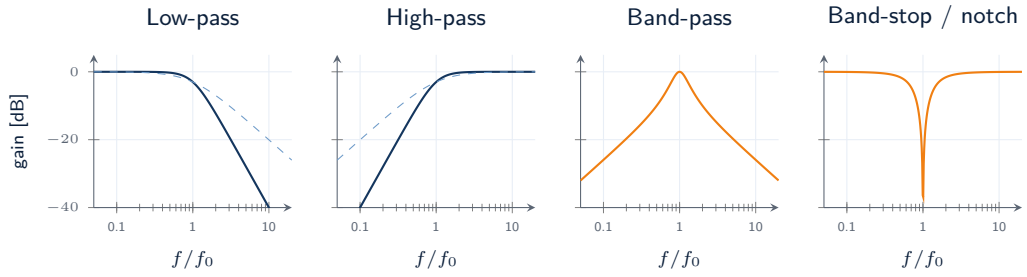
sampling frequency $f_s = 1/T_s$
Nyquist frequency $f_s/2$
aliasing fast signal disguised as slow
ADC analogue-to-digital converter
quantisation rounding to steps q
SNR signal-to-noise ratio in dB

[Oppenheim & Schafer, 2010]; [Franklin, Powell & Workman, 1998].

Session 3

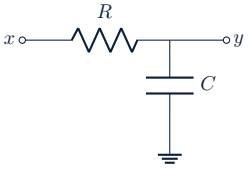
6.6 Signal processing and filters

Filter types at a glance



Type	Passes	Typical use in a drive
Low-pass	slow signals	smoothing noisy speed, anti-aliasing
High-pass	fast signals	removing offset / slow drift
Band-pass	one band	picking out one vibration frequency
Band-stop	all but one band	suppressing a resonance (notch)

From an RC low-pass to one line of code



- You know the RC circuit: y follows x with time constant $\tau = RC$; corner $f_f = 1/(2\pi\tau)$.
- In software: each sample, move y a fraction α of the way towards x:

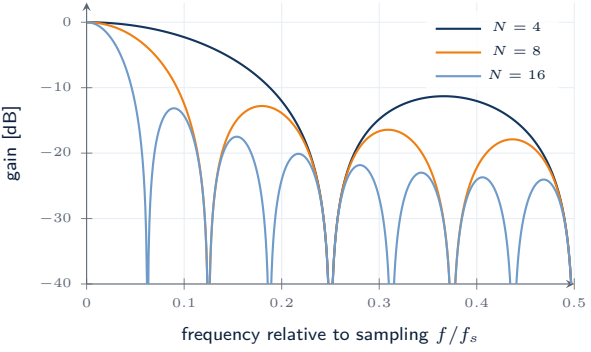
$$y[k] = y[k-1] + \alpha\,\big(x[k] - y[k-1]\big), \qquad \alpha \approx \frac{T_s}{\tau}$$

Worked example

1. Want $f_f = 500\,\text{Hz} \Rightarrow \tau = 1/(2\pi \cdot 500) = 318\,\mu\text{s}$
2. $f_s = 16\,\text{kHz} \Rightarrow T_s = 62.5\,\mu\text{s}$
3. $\alpha \approx 62.5/318 = 0.196$ (exact: $1 - e^{-T_s/\tau} = 0.178$)
4. Code: `y += 0.178f*(x - y);`

Every low-pass filter is also a **delay** of about τ (here 0.3 ms, 5 samples). Inside a loop that costs phase margin.

Moving average: the simplest digital filter



$$y[k] = \frac{1}{N} \sum_{i=0}^{N-1} x[k-i]$$

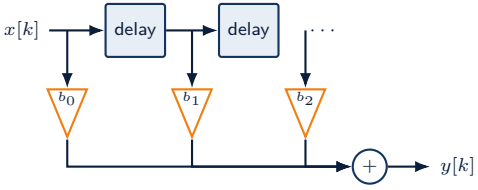
- Average of the last N samples.
- Cancels a disturbance **exactly** if its period fits the window: zeros at $f = f_s/N, 2f_s/N, \ldots$
- Delay: half the window, $\frac{N-1}{2} T_s$.
- Weak damping between the zeros.

Example

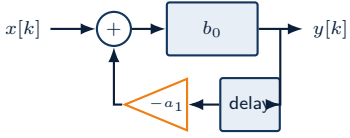
$f_s = 16\,\text{kHz}$, $N = 16$: cancels 1 kHz, 2 kHz, ...

FIR versus IIR filters

FIR: weighted past *inputs*



IIR: also feeds back past *outputs*

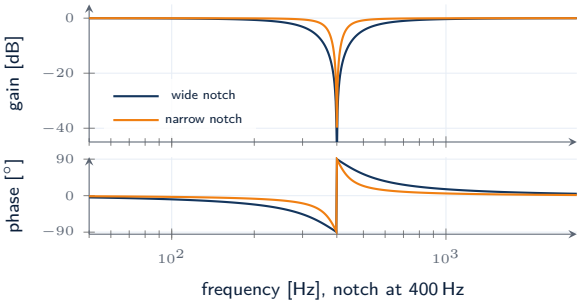


	FIR	IIR
Memory	past inputs	inputs + outputs
Stable	always	if designed so
Effort	many terms	few terms
Delay	larger	small

- Moving average = FIR. RC-type filter = IIR.
- Inside control loops **IIR** usually wins (low delay).

Going further: higher-order IIR filters are built as cascades of second-order sections ("biquads", Direct Form II transposed) to avoid rounding problems.

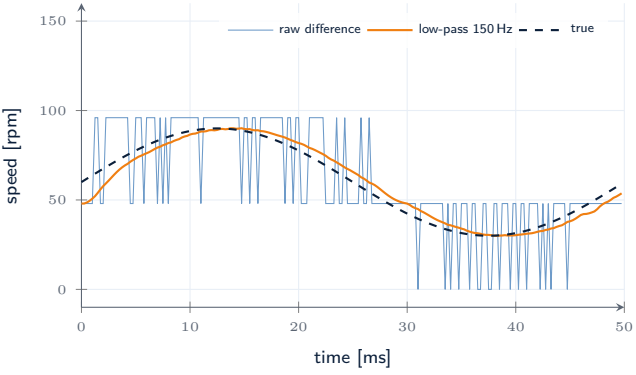
Notch filter: cutting out one robot-joint resonance



- Recall W2D1: a gearbox or a long arm makes the joint **ring** at one frequency (here 400 Hz).
- A **notch** removes a narrow band and leaves the rest almost untouched.
- Wide notch: robust if the resonance moves (payload!), but more phase loss near the loop bandwidth.
- Narrow notch: little phase loss, but must hit the frequency exactly.

> Rule: fix stiffness mechanically first; filter second.

Filtered derivative: speed from position



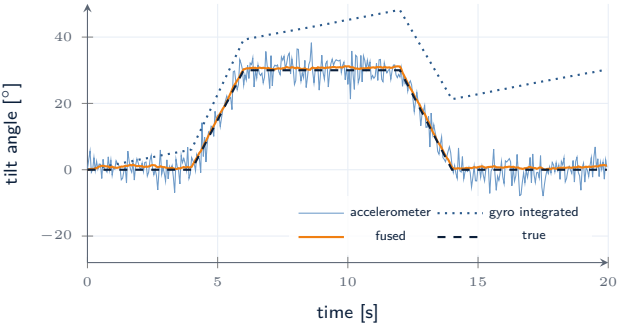
Simulation: 10 000 counts/rev, $T_s = 125\,\mu s$, true speed 60 ± 30 rpm.

- Speed $\approx \dfrac{\theta[k] - \theta[k-1]}{T_s}$: a difference of two rounded numbers.
- Dividing a one-count step by a tiny T_s gives big jumps (0, 48, 96 rpm).
- Differentiation boosts fast changes: $+20$ dB per decade.
- Fix: differentiate, then low-pass (**filtered derivative**), also used for the D-term of a PID:

$$D(s) = \frac{K_d\, s}{1 + s/\omega_f}$$

The filter adds **delay** (≈1.5 ms here).

Sensor fusion: the complementary filter



Simulation: $f_s = 100\,\text{Hz}$, gyro bias 1.5 °/s, accelerometer noise 3° rms, $\alpha = 0.98$.

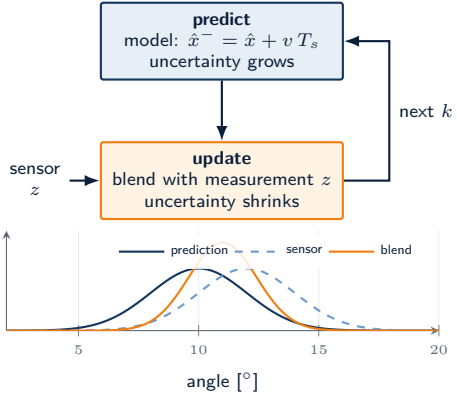
- Accelerometer: right **on average**, but noisy.
- Gyro: smooth, but integrating its small bias makes the angle **drift**.
- Trust the gyro for fast changes, the accelerometer for slow ones:

$$\hat{\theta}[k] = \alpha\big(\hat{\theta}[k-1] + \omega_g T_s\big) + (1-\alpha)\,\theta_a$$

Time constant

$\tau \approx \alpha T_s/(1-\alpha) = 0.98 \cdot 0.01/0.02 \approx 0.5\,\text{s}$.

The Kalman filter idea: blend by trust



$$\hat{x} = \hat{x}^- + K\,(z - \hat{x}^-), \qquad K = \frac{P}{P + R}$$

P: variance of the prediction; R: variance of the sensor; K: **Kalman gain**, between 0 and 1.

Worked example

1. Prediction 10°, $P = 4$; sensor 12°, $R = 4$
2. $K = 4/(4 + 4) = 0.5$
3. $\hat{x} = 10 + 0.5 \cdot 2 = 11°$, new variance $(1 - K)P = 2$

Going further: the full filter uses matrices and also estimates biases; the extended KF handles non-linear robots and AMR localisation.

Check your understanding: filters

Check your understanding

1. A moving average with $N = 8$ runs at $f_s = 8\,\text{kHz}$. Which frequencies does it cancel exactly?
2. In a complementary filter, why do we trust the gyro for fast changes but not for slow ones?
3. You need a filter inside a fast velocity loop. FIR or IIR — and why?

Summary and key terms: signal processing and filters

- Every filter changes the gain *and* adds delay; in a loop both matter.
- First-order low-pass = digital RC: one line of code.
- Moving average (FIR) cancels known periodic disturbances.
- IIR: few terms, low delay; FIR: always stable, more delay.
- Notch for one resonance; filtered derivative for speed / D-term.
- Fusion: gyro for fast, accelerometer for slow; Kalman blends by trust.

Which filter?

narrow resonance	notch
broad noise	low-pass
offset / drift	high-pass
known periodic dist.	moving avg.
two imperfect sensors	fusion / Kalman

FIR finite impulse response; **IIR** infinite impulse response; **sensor fusion** combining sensors; **Kalman gain** weight of the new measurement.

Session 4

6.7 Communication and integration

Refresher: what is a network packet?



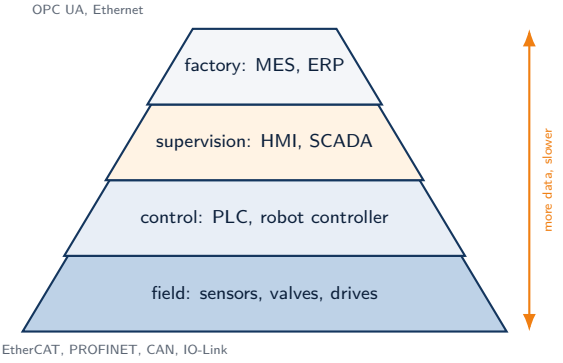


Refresher: packets

A **packet** (or **frame**) is a short block of bits sent over a cable, like a letter: an envelope with addresses, the letter inside, and a seal that shows tampering.

- **Bit rate**: bits per second on the wire.
- **Cycle time**: how often fresh packets are exchanged.
- **Latency**: how old the data is when it arrives.
- **Fieldbus**: a network built for machines, with guaranteed timing.

Networks in a robot cell: the automation pyramid



Link	Cycle
inside a drive (current, angle)	31 µs to 125 µs
controller ↔ drives, I/O	0.125 ms to 4 ms
PLC ↔ robot, camera	1 ms to 20 ms
cell ↔ HMI, MES	0.1 s to 1 s

- Low levels: small data, **hard deadlines**.
- High levels: large data, deadlines are soft.
- **Real time** means “on time, every time”, not “fast”.

CAN and CANopen

- **CAN**: two twisted wires shared by all nodes, $120\,\Omega$ resistor at each end.
- Each message has an **identifier**; if two nodes send at once, the lower identifier wins (**arbitration**) — like a polite queue by priority.
- Up to 8 data bytes per frame, 1 Mbit/s over short cables.
- **CANopen** adds a common dictionary of parameters and fast process-data messages (**PDO**).



Worked example: how fast?

1. One full frame ≈ 120 bits $\Rightarrow \approx 120\,\mu s$ at 1 Mbit/s
2. 4 axes $\times$ 2 frames $+$ 1 sync frame $= 9$ frames
3. $9 \times 120\,\mu s \approx 1.1$ ms per cycle

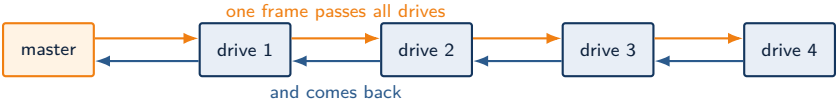
$\Rightarrow$ fine for $\geq$1 ms cycles and few axes only.



CAN interface board for a microcontroller: Seeed Wiki, CC BY-SA 4.0, via Wikimedia Commons

Going further: the CiA 402 drive profile defines one state machine and one set of objects (control word, status word, target position) so that every vendor's drive is commanded the same way over CANopen or EtherCAT.

EtherCAT: processing on the fly



- Like a train passing stations: each drive **unloads** its set-points and **loads** its actual values while the frame rushes through (well under 1 µs per drive).
- A **working counter** in the frame tells the master that every drive answered.
- **Distributed clocks**: all drives share one time base and act at the same instant.

Example: 32 axes

1. $32 \times 12\,\text{B} = 384\,\text{B} + 52\,\text{B}$ overhead
2. ≈ 3500 bit at 100 Mbit/s $\approx 35\,\mu\text{s}$
3. $+$ delays $\approx 30\,\mu\text{s} \Rightarrow$ fits a 100 µs cycle



EtherCAT slave controller IC: Allanchou1357, CC BY-SA 4.0, via Wikimedia Commons

Going further: distributed-clock details: propagation-delay measurement, drift compensation, SYNC0 events, sub-µs jitter.

PROFINET, and comparing industrial networks

Network	Speed	Typical cycle	Use in a robot cell
CAN / CANopen	1 Mbit/s	$\geq$1 ms	small robots, AMRs, grippers, sensors
EtherCAT	100 Mbit/s	62.5 µs–4 ms	robot joints, multi-axis motion
PROFINET RT	100 Mbit/s	1 ms to 10 ms	PLC $\leftrightarrow$ I/O, robot, valves
PROFINET IRT	100 Mbit/s	31.25 µs–1 ms	synchronised motion
OPC UA	Ethernet	100 ms–s	cell data to HMI / MES / cloud
ROS 2 (DDS)	Ethernet	1 ms to 100 ms	research, AMRs, vision and planning

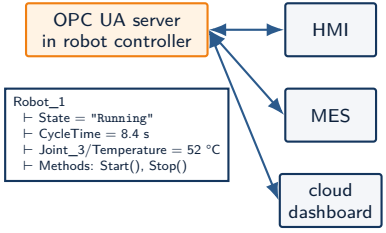
- **PROFINET**: Ethernet for PLCs; **RT** skips the slow TCP/IP layers, **IRT** reserves a fixed time slot in every cycle.
- Like a motorway with a bus lane that is closed to all other traffic.

How to choose

By **cycle time**, **synchronisation**, number of devices and what the PLC / robot vendor supports — not by bit rate alone.

Standards: ISO 11898 (CAN), IEC 61158 / 61784 (fieldbuses), IEC 62541 (OPC UA), IEC 61800-7 (drive profiles).

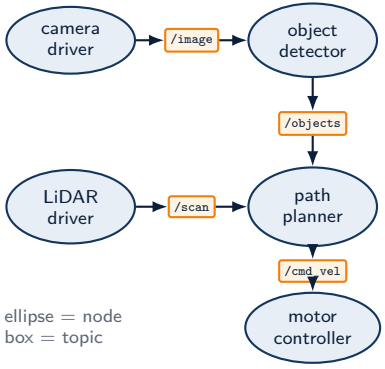
OPC UA: machines that describe themselves



- **OPC UA** = vendor-neutral way to exchange *named, typed* data, not just raw bytes.
- Each machine offers an **information model**: a tree of variables, methods and alarms with units.
- Client–server (ask when needed) or publish–subscribe (broadcast changes).
- Built-in security: certificates, encryption.
- Not for fast control loops: typical 100 ms or slower.

Going further: companion specifications (e.g. OPC UA for Robotics, VDMA 40010) define standard trees per machine type; OPC UA over TSN aims at real-time.

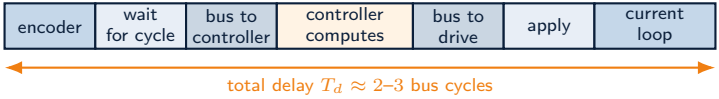
ROS: the software glue of research and mobile robots



- **ROS** (Robot Operating System) is not an OS but a **middleware**: a set of libraries and tools.
- Programs are **nodes**; they exchange messages on named **topics** (publish–subscribe), or call **services** and **actions**.
- Like a radio: anyone may broadcast on a channel, anyone may listen.
- ROS 2 uses DDS for transport; `ros2_control` links to real hardware.

Hard real-time loops (current, velocity) still run in the drive; ROS plans and coordinates.

Synchronisation and latency: which loops may run over the bus?



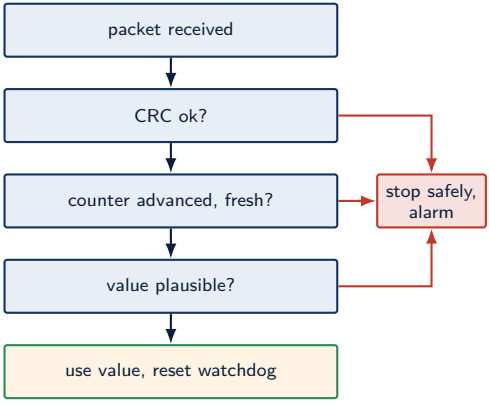
Worked example

1. Bus cycle 250 µs, $T_d = 2.5 \times 250 = 625$ µs
2. Position loop, $f_c = 30$ Hz: $360 \cdot 30 \cdot 625 \cdot 10^{-6} = 7^\circ$ ok
3. Velocity loop, $f_c = 200$ Hz: 45° too much

- Close the fast loops (current, velocity) **inside the drive**.
- Send only position set-points over the bus (CSP mode).
- Shower analogy: a long pipe (delay) makes temperature control slow — keep fast control close to the tap.
- **Synchronisation**: all axes share one clock (EtherCAT distributed clocks, IEEE 1588 PTP), so a 6-axis path stays on track.

Is the data valid? CRC, counters, watchdogs, timestamps

- **CRC**: detects bits damaged on the way. But a correct value can still be *old*.
- **Counters**: a number that increases every cycle; a frozen counter = lost data.
- **Watchdog**: a timer that trips if no fresh data arrives in time.
- **Timestamp**: age of a value $= t_{\text{now}} - t_{\text{sample}}$.
- **Plausibility**: is the value possible? e.g. $i_a + i_b + i_c = 0$.



Check your understanding: communication

Check your understanding

1. Which network would you choose (a) between robot joints, (b) between the cell and the factory MES? Why?
2. A bus has a total delay of 1 ms. How much phase does this cost a loop with $f_c = 100$ Hz? Would you close that loop over the bus?
3. A value arrives with a correct CRC but is five cycles old. Which mechanism catches this?

Summary and key terms: communication and integration

- Packets = header + data + CRC; buses exchange them every cycle.
- CAN/CANopen: robust, priority arbitration, $\geq$1 ms cycles.
- PROFINET RT/IRT for PLC cells; OPC UA for named data to MES; ROS 2 to glue robot software.
- EtherCAT: one frame through all drives, 100 µs cycles, common clock.
- Delay costs phase: close fast loops inside the drive; check freshness of data.

Key terms

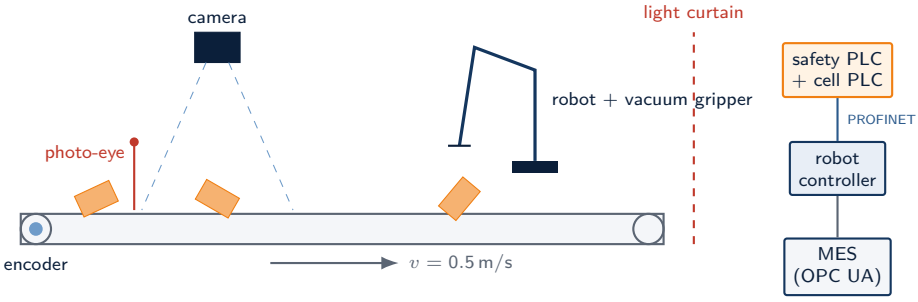
packet header + data + check
fieldbus machine network with fixed timing
OPC UA self-describing machine data
ROS topic named publish–subscribe channel
latency age of data on arrival
watchdog timer that trips on missing data

ETG.1000 / IEC 61158; IEC 62541; ISO 11898; [Quigley et al., 2009]; [Macenski et al., 2022].

Session 5

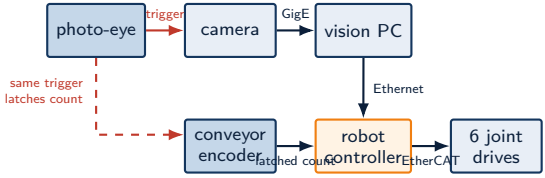
6.8 System-integration case study and course wrap-up

Case study: a vision-guided conveyor-picking cell



- Parts arrive randomly placed; the robot must pick 60 per minute.
- Today's topics meet: vision (6.4), sampling of the encoder (6.5), filtering (6.6), networks (6.7).
- Plus last week: kinematics (W2D1–2), safety (W2D4), servo loops (W1D5).
- Question: **will the gripper hit the part?**

Case study: who talks to whom, and how fast



Step	Time
trigger + exposure	2 ms
image transfer	15 ms
image processing	60 ms
send result	3 ms
total vision latency	80 ms

In 80 ms the part travels $0.5 \cdot 0.080 = 40$ mm — far more than the gripper tolerance of ± 2 mm. The answer is not a faster PC but a **time stamp**: latch the encoder count at the moment of exposure.

Case study: conveyor tracking, step by step

$$x_{\text{part}}(t) = x_{\text{image}} + \Delta n \cdot s_{\text{enc}}$$

x_{image}: position from vision at exposure; Δn: encoder counts since exposure; s_{enc}: belt travel per count [mm].

- The robot controller follows the moving part frame like a **user frame** on wheels (W2D3).
- The encoder is sampled every robot cycle; its resolution and timing set the error.

Worked example

1. Roll diameter 100 mm: circumference $\pi \cdot 100 = 314$ mm
2. Encoder 1000 counts/rev $\Rightarrow$ $s_{\text{enc}} = 0.314$ mm
3. Vision: part at $x_{\text{image}} = 120$ mm; since exposure $\Delta n = 1900$ counts
4. $x_{\text{part}} = 120 + 1900 \cdot 0.314 = 717$ mm $\rightarrow$ robot target

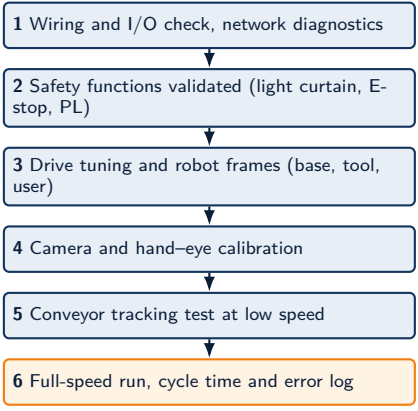
Case study: the error budget

Error source	± mm
vision: 1 px at 0.5 mm/px	0.50
hand–eye calibration	0.50
encoder: 1 count	0.31
timing jitter 1 ms at 0.5 m/s	0.50
robot repeatability	0.05
worst case (sum)	1.86
statistical (root of sum of squares)	**0.92**

$$e_{\text{RSS}} = \sqrt{e_1^2 + e_2^2 + \cdots + e_n^2}$$

- Independent errors rarely all peak together: RSS is the realistic estimate.
- Both stay below the gripper tolerance of ±2 mm: **the design works**.
- Biggest terms first: better lens or lighting, careful calibration, hardware time stamps.

Case study: commissioning the cell



- **System integration**: making separately good parts work as **one** system.
- Inside-out order: each step relies on the one before (like tuning loops inner to outer, W1D5).
- **FAT** (factory acceptance test) at the integrator, **SAT** (site acceptance test) at the customer.
- Log data with time stamps: most integration bugs are timing bugs.

Check your understanding: integration case study

Check your understanding

1. The conveyor speed doubles to $1\,\mathrm{m/s}$. Which entry of the error budget changes, and what is the new RSS value?
2. Why does latching the encoder count at the camera trigger make the vision delay harmless?
3. Why are safety functions validated *before* the robot runs at full speed?

Course wrap-up: ten days, one robot

W1D1 Motors: DC, BLDC, PMSM, stepper; electrical + mechanical models

W1D2 Gearboxes, linear actuators, motor sizing; modelling, Laplace

W1D3 Block diagrams, Bode plots, PID, stability margins

W1D4 Filters, digital control, manipulator control; drives, PWM

W1D5 FOC, cascaded loops, tuning, trajectories + feedforward

W2D1 Resonance; robot anatomy; transforms; forward kinematics

W2D2 Inverse kinematics, Jacobian, dynamics, path planning; automation

W2D3 Robot applications, work cells, PLCs, robot programming

W2D4 Material handling, safety standards; sensors I

W2D5 Vision, sampling, filters, networks, integration

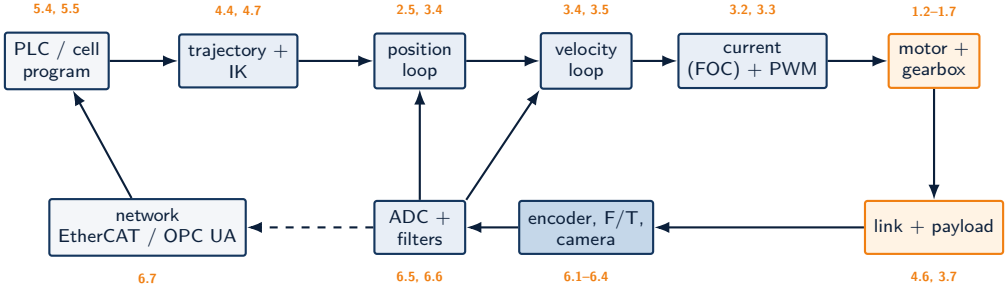
Week 1: make one joint move well

actuators → control → servo drives

Week 2: make many joints do useful work

mechanisms → manufacturing → sensors & integration

Course wrap-up: every topic in one robot joint



The course in one sentence

A robot is good when its **mechanics**, **actuators**, **control loops**, **sensors** and **networks** are designed together — the weakest link sets the performance.

Orange numbers: curriculum session codes where each block was taught.

Course wrap-up: ten principles to take home

1. Size the actuator for the *whole* motion: RMS torque, peak, heat (W1D2).
2. Model first, then measure and compare (W1D2–3).
3. Feedback needs margins: delay and filters eat phase (W1D3–4).
4. Inner loops 3–10$\times$ faster than outer loops (W1D5).
5. Feedforward does the work, feedback fixes the errors (W1D5).
6. Kinematics links joints and the tool: watch singularities (W2D1–2).
7. Design the cell around cycle time, reach and tooling (W2D3).
8. Safety is designed in from day one, not added (W2D4).
9. A measurement is only as good as its sensor, sampling and filter (W2D4–5).
10. Synchronise and time-stamp everything (W2D5).

Day summary

- **6.4** Camera model, calibration, pose, stereo, hand–eye.
- **6.5** Nyquist, aliasing, quantisation, ADC types, delay.
- **6.6** Low-pass, moving average, FIR/IIR, notch, filtered derivative, fusion.
- **6.7** CAN, EtherCAT, PROFINET, OPC UA, ROS; latency and synchronisation.
- **6.8** Conveyor-tracking cell: time stamps and an error budget.

Key terms of the day

intrinsics, **hand–eye calibration**, **disparity**, **Nyquist frequency**, **aliasing**, **quantisation**, **IIR / FIR**, **notch**, **complementary filter**, **Kalman gain**, **fieldbus**, **OPC UA**, **ROS topic**, **latency**, **time stamp**, **error budget**

Course wrap-up: where to go next

- **Control**: state-space design, observers, robust and optimal control.
- **Robotics**: dynamics and force control, motion planning, mobile robots and SLAM.
- **Vision and AI**: 3D perception, learning-based grasping.
- **Practice**: build a small robot arm or AMR with ROS 2; program a PLC; tune a real servo.

Thank you

Questions and discussion. Next time you see a robot cell, try to spot its **actuators**, its **sensors**, its **loops** and its **networks** — you now know how each of them works.



Oleg Yunakov, CC BY-SA 4.0, via Wikimedia Commons

Figure credits

All plots and diagrams not listed here are original drawings made for this course (TikZ / pgfplots; data generated by `w2d5/data/gen_data.py` and `day5/data/gen_data.py`).

File	Wikimedia Commons title	Author	License
`r25_vision_camera.jpg`	SVS Vistek EXO Tracer 8318.jpg	Ginkgomaier	CC BY-SA 4.0
`r25_checkerboard.png`	Chessboard calibration setup.png	Brian moore81	CC BY-SA 4.0
`r25_realsense.jpg`	Intel Realsense depth camera D435.jpg	Marc Auledas	CC BY-SA 4.0
`r25_mv_system.jpg`	Machine Vision System.jpg	European Editors	CC BY-SA 4.0
`r25_can_shield.jpg`	CAN BUS Shield V2.jpg	Seeed Wiki	CC BY-SA 4.0
`d4_kuka_robot.jpg`	KUKA industrial robotic arm performing “The Beach” by Roy Nachum - 02.jpg	Oleg Yunakov	CC BY-SA 4.0
`d5_ethercat_esc.jpg`	ASIX AX58101 EtherCAT SubDevice Controller.jpg	Allanchou1357	CC BY-SA 4.0

All via Wikimedia Commons (commons.wikimedia.org); licences as stated.

References

- P. Corke, *Robotics, Vision and Control*, 2nd ed., Springer, 2017.
- R. Szeliski, *Computer Vision: Algorithms and Applications*, 2nd ed., Springer, 2022.
- Z. Zhang, “A flexible new technique for camera calibration”, *IEEE Trans. PAMI* 22(11), 2000.
- R. Y. Tsai, R. K. Lenz, “A new technique for fully autonomous and efficient 3D robotics hand/eye calibration”, *IEEE Trans. Robotics and Automation* 5(3), 1989.
- A. V. Oppenheim, R. W. Schafer, *Discrete-Time Signal Processing*, 3rd ed., Pearson, 2010.
- G. F. Franklin, J. D. Powell, M. L. Workman, *Digital Control of Dynamic Systems*, 3rd ed., Addison-Wesley, 1998.
- J. Fraden, *Handbook of Modern Sensors*, 5th ed., Springer, 2016.
- B. Siciliano, O. Khatib (eds.), *Springer Handbook of Robotics*, 2nd ed., Springer, 2016.
- S. Thrun, W. Burgard, D. Fox, *Probabilistic Robotics*, MIT Press, 2005.
- M. Quigley et al., “ROS: an open-source Robot Operating System”, ICRA Workshop on Open Source Software, 2009; S. Macenski et al., “Robot Operating System 2”, *Science Robotics* 7(66), 2022.
- ISO 11898 (CAN); IEC 61158 / IEC 61784 (fieldbuses, EtherCAT, PROFINET); IEC 62541 (OPC UA); IEC 61800-7 (drive profiles); IEEE 1588 (PTP); ISO 10218 (robot safety).